

Finding similar items

Franco Reinaldo Bonifacini (41540A)

June 06, 2025

I declare that this material, which I now submit for assessment, is entirely my own work and has not been taken from the work of others, save and to the extent that such work has been cited and acknowledged within the text of my work, and including any code produced using generative AI systems. I understand that plagiarism, collusion, and copying are grave and serious offences in the university and accept the penalties that would be imposed should I engage in plagiarism, collusion or copying. This assignment, or any part of it, has not been previously submitted by me or any other person for assessment on this or any other course of study.

Abstract

This project aims to build a **detector of similar pairs of book reviews**. This report explains the full process, from data preparation to the model used. In the end, **the project returns pairs of book reviews, from a sample of 5% from the original dataset, with a Jaccard similarity of at least 80%.**

1 Introduction

This report presents the development of a similarity detection system aimed at identifying pairs of similar book reviews based on their textual content.

The report provides a detailed overview of the process, including a **short data exploration, text preprocessing (including tokenization, normalization, remotion of stopwords and lemmatization)**, and the implementation of a model that transforms tokens into vectors and finally measures its similarity. In particular, the report investigates the use of **Jaccard similarity** on sets of tokens derived from user reviews, while also exploring text representations such as **shingles, HashingTF and MinHashingLSH**. These choices impact both the accuracy and computational efficiency of the similarity detector.

To evaluate the performance of the system, a subset of the data is used to validate the quality of the detected review pairs, with discussion on **threshold selection** and **scalability**.

2 Dataset Used and Pre-Processing

2.1 Dataset

The dataset used in this project was obtained from **Kaggle** [1]. There were two files available in the source, but for this project, only the one called **Books_rating** was used. This dataset contains **3 million records**, with information about the books, the users who reviewed them, and the reviews themselves.

2.2 Dataset's Attributes

This dataset contains 10 columns which are:

- **Id (object)**: The ID of the book.
- **Title (object)**: The title of the book.
- **Price (float)**: The price of the book.

- **User_id (object)**: The ID of the user who wrote the review.
- **profileName (object)**: The name of the user's profile.
- **review/helpfulness (object)**: Helpfulness rating of the review.
- **review/score (float)**: The score the user gave the book.
- **review/time (int)**: The time when the review was written.
- **review/summary (object)**: A short summary of the review.
- **review/text (object)**: The full text of the review.

2.3 Data Consideration

During the project, **some reviews were found to be duplications (based on the project's purpose)**. These were cases where the same user wrote identical reviews for the same book. Although other values such as helpfulness or time were different, they were treated as duplicates and removed. As a result, **380,652 rows were removed, leaving a dataset with 2,619,348 rows, which represent 87.31% of the original data.**

Furthermore, only **Id, Title and review/text** were kept for the project, and missing values were analyzed.

```
Id 0
Title 208
review/text 8
```

```
dtype: int64
```

Since the missing values were less than 1% of the data, they were removed. This final cleaning step **removed 216 rows, resulting in a dataset with 2,619,132 rows, which represents 87.3% of the original data.**

Last but not least, **5% of the dataset** was selected for the implementation phase of the project. This allowed for enough data to test the model while also ensuring that the final output could be generated in a reasonable amount of time.

2.4 Pre-processing Techniques

To prepare the text data for similarity detection, a **Natural Language Processing (NLP) pipeline** was applied to the review/text column. A Pipeline consists of a sequence of stages, each of which is a Transformer [2]. In this case, the pipeline was implemented through the function `text_pipeline(df, input_col = 'review/text')`.

This pipeline performs all necessary text transformations and **produces a final column containing the tokens that will be used in the model:**

- **Document Assembling**: Prepares data into a format that is processable by Spark NLP [3].
- **Tokenization**: Each review was split into individual tokens (words) [4].
- **Normalization**: Tokens were transformed to lowercase and cleaned by removing punctuation and non-alphanumeric characters [5].
- **Stopword**: process of eliminating words that occur frequently in a language but carry little or no meaning [6].
- **Lemmatizer**: Words were reduced to their base or root form (for example: "running" becomes "run") [7].
- **Finisher**: The processed tokens were converted into a plain format, stored in a new column called `final_tokens` [8].

As a result, the pipeline outputs a new column containing a cleaned array of lemmatized tokens from the original reviews. Including **lemmatization helps reduce redundancy and improve semantic accuracy**, as different word forms with the same meaning are unified. This pre-processed column is then used in the final model to detect similar items.

First 10 rows from the dataset were outputted:

Id	review/text	final_tokens
B000NTMPGK	Of all the James...	[james, patterson...]
0375828095	Another story in...	[another, story,...]
0809123320	Again, I seem to...	[seem, purchase,...]
073933252X	I read this book...	[read, book, firs...]
B000GK17RY	This was a great...	[great, book, sha...]
0972847227	Just received min...	[receive, i, orig...]
1841010162	Great book! Very...	[great, book, pro...]
1565791541	The photography w...	[photography, love...]
0061065560	Before I ever kne...	[ever, know, book...]
B000OVGENM	Excellent book! D...	[excellent, book,...]

3 Algorithms and Scalability

This section describes the model used to identify pairs of similar items and how it can scale with increasing data size. The process begins by transforming the **final_tokens** into **shingles**, which are then **hashed** into numerical values for efficient comparison. These hashed values are used to generate **vectors using MinHashLSH**. Finally, item similarity is computed using **the Jaccard similarity**, and each item is paired with its most similar review.

3.1 Shingles

To efficiently identify lexically similar documents, a common strategy is to represent each document as a set of substrings called **shingles**. A **k-shingle** is defined as any contiguous sequence of k characters extracted from a document. This transformation allows us to capture the local structure of the text, such that even if two documents present the same content in different word orders, they may still share a significant number of shingles [11].

The choice of k is critical. A very small k may result in many unrelated documents appearing similar, since short substrings are more likely to occur by chance. Conversely, a very large k might miss meaningful overlaps between similar documents. As a rule of thumb, k should be large enough to reduce the chance of a given shingle appearing in unrelated documents, but small enough to capture shared phrases across related texts.

For this project it was used a **k=3** as it takes into account the pre-processed tokens which already removed whitespaces and any stopwords. Although a higher k as 4 or even 5 may be acceptable, for time optimization reasons it was set to 3. Considering this, an additional filter was done to the dataset to prevent future errors with empty vectors. So every case with final_tokens lower than 3 was removed.

```
filtered_pipeline_result = pipeline_result.filter(size(col("final_tokens")) >= 3)
```

For this step, it was used PySpark's NGram transformer to create sequences of three consecutive words from the tokenized text stored in the final_tokens column. By setting n=3, it was ensure that each document is transformed into a list of word triples that capture short phrases within the text. The output was stored in a new column called **shingles**. The following code was used to extract 3-gram shingles from the filtered tokens:

```
ngram = NGram(n=3, inputCol="final_tokens", outputCol="shingles")
shingled_df = ngram.transform(filtered_pipeline_result)
```

Finally a new column with the shingles was obtained, a sample of the first 10 rows was outputted:

Id	final_tokens	shingles
B000NTMPGK	[james, patterson...]	[james patterson...]
0375828095	[another, story,...]	[another story se...]
0809123320	[seem, purchase,...]	[seem purchase bo...]
073933252X	[read, book, firs...]	[read book first,...]
B000GK17RY	[great, book, sha...]	[great book share...]
0972847227	[receive, i, orig...]	[receive i origin...]
1841010162	[great, book, pro...]	[great book provo...]
1565791541	[photography, love...]	[photography love...]
0061065560	[ever, know, book...]	[ever know book,...]
B000OVGENM	[excellent, book,...]	[excellent book d...]

3.2 Hashing Shingles

Instead of storing each shingle as a full substring, it was applied a **hash function** that maps each k-shingle to a fixed-size integer, typically by assigning it to one of many possible buckets. This technique reduces the memory needed to represent each shingle and allows faster comparisons using integer operations [11].

For this project, the parameter **numFeatures** was set to **2048** which defines the dimensionality of the hashed feature space. It balances performance and memory usage by reducing the likelihood of hash collisions while keeping the representation compact. Since **2048 is a power of two, it is also computationally efficient**.

The following transformation was applied to hash the shingles into fixed-length feature vectors:

```
# Define the transformer
hashing_tf = HashingTF(inputCol="shingles", outputCol="hashed_shingles", numFeatures=2048)

# Apply the transformation
hashed_shingles_df = hashing_tf.transform(shingled_df)
```

After the implementation of this algorithm, it a new column was obtained with the hashed shingles, and a sample of the first 10 rows was outputted:

Id	shingles	hashed_shingles
B000NTMPGK	[james patterson...]	(2048,[133,206,24...
0375828095	[another story se...]	(2048,[62,152,163...
0809123320	[seem purchase bo...]	(2048,[31,42,71,9...
073933252X	[read book first,...]	(2048,[39,61,64,1...
B000GK17RY	[great book share...]	(2048,[171,448,60...
0972847227	[receive i origin...]	(2048,[489,539,77...
1841010162	[great book provo...]	(2048,[27,144,170...
1565791541	[photography love...]	(2048,[123,213,23...
0061065560	[ever know book,...]	(2048,[25,195,208...
B000OVGENM	[excellent book d...]	(2048,[53,60,68,7...

3.3 MinHashLSH

Finally, to efficiently compare large sets and identify similar ones, we can represent collections of sets as characteristic matrices, where rows correspond to elements from a universal set and columns represent individual sets. A 1 in a matrix cell indicates membership of an element in a set. Since these matrices are often sparse, they are not stored explicitly; instead, we use compact representations like **MinHash signatures**. A MinHash signature is constructed by applying multiple random permutations (using hash functions in this case) to the rows and recording the first row with a 1 in each case.

Nevertheless, to avoid the computational cost of comparing all set pairs, **Locality-Sensitive Hashing (LSH)** is used. LSH hashes MinHash signatures into buckets in such a way that similar sets are more likely to fall into the same bucket, significantly reducing the number of comparisons [11].

```
# Define the model
minhash = MinHashLSH(inputCol="hashed_shingles", outputCol="minhashes", numHashTables=4)

# Fit the model to the data
minhash_model = minhash.fit(hashed_shingles_df)

# Transform the input data and cache results
minhashed_df = minhash_model.transform(hashed_shingles_df).cache()
```

Additionally, `.cache()` was implemented to avoid recalculation of MinHashLSH result every time is called [9].

So at the end, a new column was obtained which will be used for finding similar items, and a sample of the first 10 rows was outputted:

Id	hashed_shingles	minhashes
B000NTMPGK	(2048,[133,206,24...	[[3.6290422E7], [...
0375828095	(2048,[62,152,163...	[[4.8992447E7], [...
0809123320	(2048,[31,42,71,9...	[[1.9594551E7], [...
073933252X	(2048,[39,61,64,1...	[[1.0339328E7], [...
B000GK17RY	(2048,[171,448,60...	[[5.461852E7], [3...
0972847227	(2048,[489,539,77...	[[3.33898532E8], ...
1841010162	(2048,[27,144,170...	[[1.61140306E8], ...
1565791541	(2048,[123,213,23...	[[9.6718441E7], [...
0061065560	(2048,[25,195,208...	[[5.2803945E7], [...
B000OVGENM	(2048,[53,60,68,7...	[[5.6250747E7], [...

3.4 Jaccard Similarity

Jaccard similarity is a commonly used metric for **comparing the similarity between two sets**. It is defined as the size of the intersection divided by the size of the union of the sets. The resulting value ranges from 0 to 1, where 0 indicates no overlap, and 1 means the sets are identical. This measure is especially useful in applications such as document similarity, clustering, duplicate detection, and when working with binary attributes. It provides an intuitive and mathematically grounded way to quantify how much two sets have in common. Formally, for two sets A and B , the Jaccard similarity is defined as [11]:

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|}$$

In this project, an **approximate similarity join** was applied using the results from MinHashLSH. This allowed identifying pairs of rows with a Jaccard similarity of at least 80%. Since Spark's 'approx-SimilarityJoin' uses Jaccard *distance*, and not similarity, it is important to recall the relation:

$$\text{Jaccard Distance} = 1 - \text{Jaccard Similarity}$$

Hence, setting the threshold to 0.2 ensures that only pairs with a similarity of at least 0.8 (80%) are returned.

To avoid redundant comparisons, two filters were applied:

- Only pairs where `datasetA.id < datasetB.id` are included, to avoid duplicate or reversed matches.
- Pairs with identical review texts are excluded, using `datasetA.review/text != datasetB.review/text`.

All things considered, the final result was obtained and a sample of the first 10 rows was outputted:

doc_id	review/text	most_similar_doc_id	review/text	jaccard_distance
0708982581	Emma Woodhouse, "...	8437615607	Emma Woodhouse, "...	0.156
0395051150	Emma Woodhouse, "...	8437615607	Emma Woodhouse, "...	0.156
0340283947	The BLTC Kindle e...	1582343632	This Kindle editi...	0.1058
B0000YSH5G	this is the BEST ...	B000GQK706	The best book I h...	0.0
0904724719	This Kindle editi...	1582343632	This Kindle editi...	0.06
B000BI4160	"The Lord of the ...	B000H7EO2G	"The Lord of the ...	0.0
B0006AL5RG	I just wanted to ...	B000J6DLBU	I just wanted to ...	0.03546
0945466307	gave it as a gift...	140004796X	gave it as a gift...	0.0
1561797863	Prince Caspian: T...	B000I12TCS	Prince Caspian: T...	0.0
B0000B194M	Note: Letter grad...	B0008CVIEW8	Note: Letter grad...	0.064257

Although it seems to be the same review, it was checked that it corresponds to different books id. Also it was discovered that some users wrote the exact same review into other books, this is why it may seem confusing the results shown, but apart from these cases explained, it is also visible some different cases with a good Jaccard distance.

3.5 Scalability

This project was built with scalability in mind, making use of **PySpark's** ability to handle large datasets through distributed computing. Thanks to **Apache Spark's architecture**, the data is automatically split and processed across **different nodes in a cluster**, which allows operations like shingling, MinHashing, and approximate similarity joins to run in parallel and more efficiently as the dataset grows [10].

One of the advantages of this implementation is its flexibility. During development, a small sample of the dataset was used for testing and faster iterations. However, scaling to the full dataset—or even larger ones—is straightforward. By simply changing the input data path or modify a sample parameter, the same code can run on a local machine or on a Spark cluster without needing major modifications.

Additionally, features like caching intermediate results (using `.cache()`) help reuse data in memory and reduce redundant computations. This becomes particularly useful when working with expensive operations like similarity joins.

4 Conclusion

This project was created to be done in a short period of time and get enough results to test the model. Considering this, by using a sample from the original dataset, the thresholds selected (k-shingles, numfeatures for hashing and numHashTables for MinHashLSH) outputted results that were considered reasonable and good enough to test the model.

The final result was validated with individual cases to check if there was no errors when pairing items, and this is why it was considered correct. Further tests are highly recommended, mainly if the dataset used is increased in size. Thresholds may be modified, but knowing that computing time and complexity may increase as model may become more accurate.

References

- [1] Mohamed Bekheet, "Amazon Books Reviews". Kaggle, 2022. <https://www.kaggle.com/datasets/mohamedbakhet/amazon-books-reviews>
- [2] "Pipeline". Apache Spark, n.d. <https://spark.apache.org/docs/latest/api/python/reference/api/pyspark.ml.Pipeline.html>
- [3] "sparknlp.base.document_assembler". John Snow LABS, n.d. https://sparknlp.org/api/python/reference/autosummary/sparknlp/base/document_assembler/index.html
- [4] David Cecchini, "Unleashing the Power of Text Tokenization with Spark NLP". John Snow LABS, Jan. 17, 2024. <https://www.johnsnowlabs.com/unleashing-the-power-of-text-tokenization-with-spark-nlp/>
- [5] Gursev Pirge, "Text Cleaning: Standard Text Normalization with Spark NLP". John Snow LABS, Jun. 07, 2023. <https://www.johnsnowlabs.com/text-cleaning-standard-text-normalization-with-spark-nlp/#:~:text=In%20NLP%2C%20a%20Normalizer%20is,will%20provide%20TOKEN%20as%20output.>
- [6] Gursev Pirge, "Text cleaning: removing stopwords from text with Spark NLP". John Snow LABS, Jun. 14, 2023. <https://www.johnsnowlabs.com/text-cleaning-removing-stopwords-from-text-with-spark-nlp/>
- [7] "sparknlp.annotator.lemmatizer". John Snow LABS, n.d. <https://sparknlp.org/api/python/reference/autosummary/sparknlp/annotator/lemmatizer/index.html>
- [8] "sparknlp.base.finisher". John Snow LABS, n.d. <https://sparknlp.org/api/python/reference/autosummary/sparknlp/base/finisher/index.html>
- [9] "pyspark.sql.DataFrame.cache". Apache Spark, n.d. <https://spark.apache.org/docs/latest/api/python/reference/pyspark.sql/api/pyspark.sql.DataFrame.cache.html>
- [10] "Apache Spark - A Unified engine for large-scale data analytics". Apache Spark, n.d. <https://spark.apache.org/docs/latest/>
- [11] A. Rajaraman & J. Ullman, "Mining of Massive Datasets". n.d. <http://infolab.stanford.edu/~ullman/mmds.html#latest>